# RELEASE FLOW SETUP README

Table of Contents


***A. OVERVIEW*** ..... ***2***

1. Overview – How the Release Framework Works ..... 2
2. Script Pairing Concept ..... 2

***B. GLOBAL RELEASE SCRIPT MAINTAINER GUIDE*** ..... ***2***

Core Responsibilities ..... 2

1. Maintain Central Release Path ..... 2
2. Protect Directory Structure Logic ..... 2
3. Maintain Mandatory Release Questions ..... 2
4. Validate File Copy Logic ..... 2
5. Permission and Access Control ..... 2
6. Version Control & Change Management ..... 2

**DOs** ..... **2**

**DON'Ts** ..... **3**

**Risk Areas to Monitor** ..... **3**

***C. Final Responsibility Principle*** ..... ***3***

***D. LOCAL RELEASE SCRIPT MAINTAINER GUIDE*** ..... ***3***

***PAGE 1 — LIB/LEF Release Flow*** ..... ***3***

**Purpose** ..... **3**

**Step 1 — Edit Local Script** ..... **3**

**Step 2 — Run Script** ..... **3**

***PAGE 2 — PV/IR Release Flow*** ..... ***4***

**Purpose** ..... **4**

**Step 1 — Edit Local Script** ..... **4**

***PAGE 3 — Best Practices & Guidelines*** ..... ***5***

## A. OVERVIEW

### 1. Overview – How the Release Framework Works

This release framework standardizes block releases across stages, ensures timestamp validation, auto-creates structured release directories, captures mandatory release notes, and copies validated data to a central release area.

Flow Architecture:

User Area (Local Input Script) → Global Script (Central Logic) → Central Release Area

### 2. Script Pairing Concept

Each release stage has a Local Script (user editable) and a Global Script (central logic).

LIB/LEF Stage: lef_lib_inputs.sh → global_release_script_lib_lef.sh

PV/IR Stage: pv_ir_inputs.sh → global_release_script_pv_ir.sh

…

…

Rule: Users edit ONLY the local script. The global script must not be modified.

## B. GLOBAL RELEASE SCRIPT MAINTAINER GUIDE

The Global Script Maintainer is the custodian of release governance. While users update local scripts, the global script enforces structure, ensures compliance, protects release integrity, and maintains auditability.

### Core Responsibilities

### 1. Maintain Central Release Path

Ensure central_release_dir path exists, has proper permissions, sufficient storage, and is backed up regularly. Communicate any path changes to all teams.

### 2. Protect Directory Structure Logic

Do not modify folder hierarchy creation, timestamp-based naming, or stage naming without ecosystem review. These impact automation and audits.

### 3. Maintain Mandatory Release Questions

Add new signoff checks only if universally required. Maintain backward compatibility and avoid unnecessary stage-specific additions.

### 4. Validate File Copy Logic

Ensure correct extensions are copied, avoid wildcard overreach, and prevent silent failures if variables are empty. Implement fail-fast checks for missing files.

### 5. Permission and Access Control

Ensure correct directory permissions, prevent deletion of older releases, and keep the global script read-only for general users.

### 6. Version Control & Change Management

Maintain version header, modification date, change log, and increment version for every update. Test changes in sandbox before production rollout.

### DOs

- Keep script generic and stage-agnostic.

- Add defensive checks for missing files.
- Test in sandbox before updating production.
- Maintain backward compatibility.
- Document every change.
- Keep logs verbose and informative.
- Preserve timestamp validation and overwrite protection.
- Validate disk usage periodically.

### DON'Ts

- Do not hardcode project-specific paths.
- Do not remove timestamp confirmation step.
- Do not bypass user confirmation.
- Do not auto-release without validation.
- Do not silently ignore missing files.
- Do not allow overwriting previous releases.
- Do not change folder structure without review.
- Do not give write access of global script to general users.

### Risk Areas to Monitor

- Storage exhaustion.
- Permission corruption.
- Script tampering.
- Inconsistent naming conventions.
- Incomplete release notes.
- Tool upgrade impacts.

## C. Final Responsibility Principle

The local user controls WHAT is released. The global maintainer controls HOW it is released. Treat the global script as production infrastructure.

## D. LOCAL RELEASE SCRIPT MAINTAINER GUIDE

### PAGE 1 — LIB/LEF Release Flow

### Purpose

Used to release .lib files and the .lef file. Stage folder name: LIB_LEF

### Step 1 — Edit Local Script

Open lef_lib_inputs.sh and update:

```
set block_name = "your_block_name"

set block_lib_dir = "/path/to/lib_directory"

set block_lef = "/path/to/block.lef"
```

### Step 2 — Run Script

```
csh lef_lib_inputs.sh
```

### *Internal Flow*
Script prints timestamps and asks for confirmation (y/n).

If 'y' → release proceeds. Otherwise → exits safely.

### *Directory Structure Created*
RELEASE_AREA/block_name/DD-Month-YY_HHMM/LIB_LEF/

Contains: LIB/, LEF/, RELEASE_NOTE

### *Mandatory Release Notes*
Release Comments

Version of write_data_global_script

### *Files Copied*
*.lib → LIB/

block.lef → LEF/block_name.lef

## PAGE 2 — PV/IR Release Flow

### Purpose
Used for signoff-quality release including DEF, LEF, Netlists, LVS Netlist, GDS, Hollow GDS, SDC, View Definition, SPEF, PnR DB, and Memory List.

Stage folder name: PV_IR

### Step 1 — Edit Local Script
Open pv_ir_inputs.sh and update all required paths:

block_name, block_lib_dir, sta_sdc_path, sta_view_defn, block_lef, block_def,

block_pnr_db, block_pnr_netlist, block_pnr_lvs_netlist, block_spef_dir,

block_gds, block_hollow_gds, all_mem_list, block_pnr_pg_netlist

### *Important Notes*
Ensure SPEF directory contains *.spef.gz files.

Update GDS merge list path correctly.

Provide memory dump path if applicable.

### *Step 2 — Run Script*
csh pv_ir_inputs.sh

### *Directory Structure Created*
RELEASE_AREA/block_name/DD-Month-YY_HHMM/PV_IR/

Subfolders: LIB, SDC, GDS, DEF, LEF, PNR, SPEF, NETLIST

### *Mandatory Signoff Questions*
Release Comments

Script Version

IR/EM/STA Release Confirmation

PnR Shorts Clean

LVS Clean

DRC Count

LEC Clean

Dummy Metal Done

FILL & DECAP Done

Hold ARC Status

### *Files Copied*

LIB files → LIB/

Netlists → NETLIST/

LEF → LEF/

DEF → DEF/

SDC + viewDefinition → SDC/

PnR DB → PNR/

SPEF → SPEF/

GDS + Hollow GDS → GDS/

## PAGE 3 — Best Practices & Guidelines

### *Pre-Release Checklist*

All signoff checks completed

Correct netlist version used

Latest DEF verified

SPEF regenerated after ECO

LVS clean

DRC acceptable

Timestamps verified

### *Common Mistakes to Avoid*

Forgetting to update block_name

Wrong SPEF directory

Empty block_lib_dir

Not validating timestamps

## *Safety Features*

Timestamp validation before release

Manual confirmation required

Mandatory release notes

Structured directory creation

No overwrite due to timestamped folders

## *Summary*

This framework ensures repeatability, traceability, controlled signoff flow, centralized storage, and auditable release notes. It works across any block, project, PnR run, and signoff stage.